

Specification-Aligned Repair for EvalPlus Semantic Failures: Data Infrastructure Verification and Pre-Registered Experimental Design

Anonymous Authors¹

Abstract

Static analysis tools (ruff, mypy) detect at most 32.4% of the code generation failures produced by GPT-4o-mini on the EvalPlus benchmark — the remaining errors are semantic divergences that evade syntactic oracles and are resistant to blind reprompting. This paper proposes *specification-aligned repair*, a structured oracle that simultaneously provides the model with three components: the problem docstring’s formal intent, a failing test’s input/expected-output pair, and the model’s actual incorrect output. To enable controlled evaluation of this approach, a pre-experiment existence verification is conducted on the 134-problem GPT-4o-mini EvalPlus failure set from a prior round-0 evaluation (34 HumanEval+ and 100 MBPP+ failures). Verification confirms that all 134 failure identifiers, stored incorrect solutions, EvalPlus API access, and deterministic test selection are available (4/4 conditions satisfied; 5/5 pytest tests pass in 2.32 seconds). Six tasks with empty archived failing-test records define a 128-task working set with sufficient statistical power for the planned mechanism experiments. This paper constitutes a Stage 1 infrastructure and pre-registration report: three predictions are registered — that specification-aligned repair outperforms blind reprompting (P1), that it achieves a fix rate of at least 15% above the round-0 baseline (P2), and that HumanEval+ fix rate exceeds MBPP+ fix rate (P3, exploratory) — prior to any mechanism data collection. Comparative results are forthcoming from pre-registered experiments h-m1, h-m2, and h-c1.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Large language models achieve strong aggregate pass rates on code generation benchmarks, yet a non-trivial fraction of failures persist after round-0 evaluation. On EvalPlus — a benchmark that augments the base HumanEval and MBPP test suites by approximately 80× — GPT-4o-mini achieves round-0 pass@1 of 79.3% on HumanEval+ and 73.5% on MBPP+, leaving 34 and 100 failures, respectively. These failures are not stylistic or syntactic errors. A systematic analysis using the ruff linter and mypy type checker demonstrates that static analysis fires on only 32.4% of HumanEval+ failures and 16.0% of MBPP+ failures. Two-thirds to five-sixths of failures receive no actionable diagnostic signal from the tools that constitute the standard first-pass repair pipeline.

The dominant failure mode is semantic divergence: the generated code is syntactically valid and type-consistent, but algorithmically incorrect. The model has implemented the wrong algorithm or mishandled a class of edge cases. Asking it to try again without additional information — blind reprompting — provides minimal repair signal, because the model lacks evidence that its prior solution was wrong or why it was wrong.

Targeted algorithmic repair requires three distinct pieces of information simultaneously. First, the model needs to know what algorithm it was intended to implement: the formal intent captured in the problem docstring. Without this re-anchoring, the model may regenerate the same incorrect solution. Second, the model needs a concrete behavioral gap: a failing test case that specifies an input and the expected output, establishing exactly where the model’s behavior diverges from the specification. Third, the model needs a deviation signal: its own actual incorrect output, which enables it to compare its behavior against the expected output and identify the locus of the algorithmic error. Each component addresses a distinct failure of diagnostic information; no single component is sufficient.

This structured triple — problem docstring’s formal intent, failing test input/expected-output pair, and actual model output — constitutes what the present work calls *specification-aligned repair*. The design is motivated by converging evidence across independent lines of research. Specification grounding improves correct code generation by 38 percentage points, with docstring context identified as the primary driver (Haeri & Ghelichi, 2026). Removing docstring context from repair feedback causes severe performance degradation (Dai et al., 2025). Contrastive input/output pairs outperform single failure messages in automated program repair, fixing 143 of 337 bugs against 124 with single failure messages (Kong et al., 2024). Content — including actual incorrect output — rather than mere re-exposure drives LLM repair improvement, with a measured effect of +18 percentage points over bare code re-exposure ($p = 0.00042$; Iscan, 2026).

Despite this converging literature, no prior work has directly tested specification-aligned repair against blind reprompting on EvalPlus’s own semantic failure population, using a controlled three-condition paired design with pre-registered predictions. This gap is consequential: EvalPlus’s augmented test suite is substantially stricter than benchmarks used in prior repair work, and the GPT-4o-mini round-0 failure set represents a natural, unselected population of semantic errors rather than a curated bug corpus.

The present paper reports the infrastructure and pre-registration stage of a two-stage research plan. The infrastructure stage establishes and verifies the data foundations required to execute the mechanism experiments without new baseline API calls. The pre-registration stage commits three predictions before any mechanism data are collected, following the pre-registered open science tradition (?). The infrastructure and pre-registration are reported jointly as a complete scholarly contribution to reproducibility and transparency in LLM repair research.

The contributions of this paper are as follows:

Data infrastructure verification. The 134-problem GPT-4o-mini EvalPlus failure set from h-e1 Run 2 is verified as fully recoverable from archive across four conditions: all 134 failure identifiers are confirmed (C1), all 134 stored incorrect solutions are present in `solutions\_cache.jsonl` (C2), the EvalPlus API is accessible for all 134 task identifiers (C3), and deterministic test selection via `plus\_input[0]` is confirmed with 780 augmented test cases for a sample task (C4). All four conditions pass; 5/5 pytest integration tests pass in 2.32 seconds. Six tasks with empty `plus\_fail\_tests` fields define a

128-task working set representing 95.5% recovery.

Pre-registered three-condition comparative design. Three predictions are registered prior to mechanism experiment execution: (P1) specification-aligned repair achieves significantly higher round-1 pass@1 than blind reprompting (McNemar one-tailed $p < 0.05$); (P2) specification-aligned repair achieves significantly higher pass@1 than the round-0 baseline with fix rate at least 15%; (P3, exploratory) the HumanEval+ fix rate exceeds the MBPP+ fix rate under specification-aligned repair.

Methodological contribution. The four-condition existence verification protocol constitutes a reusable template for pre-experiment reproducibility checks in LLM repair research, enabling repair experiments on archived failure sets without new baseline API calls.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the methodology. Section 4 specifies the experimental setup. Section 5 presents the verification results and pre-registered predictions. Section 6 discusses implications and limitations. Section 7 concludes.

2. Related Work

This work sits at the intersection of three research areas: LLM self-repair with feedback oracles, specification grounding for code generation, and statistical design for paired LLM repair evaluation.

2.0.1. 2.1 LLM SELF-REPAIR AND FEEDBACK ORACLES

Self-Refine (Madaan et al., 2023) established that iterative self-generated feedback improves performance by approximately 20 percentage points across multiple tasks. The Self-Refine setting conflates re-exposure with informative self-generated feedback, making it difficult to isolate the contribution of feedback content from the contribution of additional sampling. The blind reprompting condition in the present design (Condition B) reduces Self-Refine’s iterative self-feedback to a single fixed reprompt string, isolating re-exposure from informative content.

FeedbackEval (Dai et al., 2025) provides the closest systematic study of feedback type effects on code repair, finding that mixed feedback achieves a 63.6% fix rate compared to 53.1% for minimal feedback on a purpose-built benchmark. Critically, removing docstring context from feedback prompts causes severe performance degradation, providing direct empirical motivation for the docstring component of the specifi-

cation triple. However, FeedbackEval does not isolate specification context from blind re-exposure through a paired design, nor does it evaluate on EvalPlus.

Arimbur (2026) evaluates iterative self-repair on HumanEval across model scales, finding improvement of 4.9 to 17.1 percentage points over several rounds, with assertion errors proving hardest at approximately 45% failure rate. This confirms that EvalPlus-style failures are difficult for self-repair but does not test structured specification context as an oracle.

The DUALFIX pipeline (Akli et al., 2026) demonstrates that execution-feedback repair leaves a residual failure class addressable only by specification-level intervention, with evolved natural language repair rules fixing 12–17% of cases that execution repair cannot address. This finding directly supports the claim that semantic-level context is necessary beyond error traces.

2.0.2. 2.2 SPECIFICATION GROUNDING FOR CODE GENERATION

Haeri and Ghelichi (2026) show that specification grounding improves correct code generation by 38 percentage points, with docstring context identified as the primary driver. Although the result is on code generation rather than code repair, the underlying mechanism — docstring re-anchoring preventing goal mis-specification — transfers directly to the repair setting, where the model may regenerate the same incorrect solution without explicit re-anchoring on the intended algorithm.

ContrastRepair (Kong et al., 2024) tests contrastive input/output pairs for automated program repair on the Defects4J corpus, finding 143 of 337 bugs fixed with contrastive pairs compared to 124 with single failure messages. The contrastive pair design — providing expected output alongside actual output — is directly analogous to the input/output pair component of the specification triple. The key difference is that ContrastRepair targets Java bugs in a classical automated program repair setting, not LLM code generation on EvalPlus, and does not include docstring re-anchoring as a third component.

VRpilot (Kulsum et al., 2024) applies chain-of-thought reasoning with patch validation to vulnerability repair, finding 14% more correct patches than naive repair. The consistent advantage of structured semantic context over minimal feedback across repair domains provides convergent support for the present design.

SGCR (Wang et al., 2025) validates specification-grounded code review at industry scale, with a 42%

developer adoption rate and 90.9% improvement in review utility, establishing that specification grounding has practical value in production code assessment settings.

2.0.3. 2.3 STATISTICAL DESIGN FOR LLM REPAIR EVALUATION

Iscan (2026) provides the methodological foundation for the McNemar design used here. The paper validates placebo-controlled decomposition of self-repair feedback using frozen small code models, demonstrating that content (code plus facts) outperforms bare code re-exposure by 18 percentage points ($p = 0.00042$). The paper explicitly validates the one-tailed McNemar design for paired LLM repair comparisons — the same design pre-registered in the present work for predictions P1 and P2.

The EvalPlus benchmark (Liu et al., 2023) is the evaluation environment for this work. Its $80\times$ test augmentation over base HumanEval produces a substantially stricter evaluation that specifically surfaces semantic divergences surviving round-0 evaluation — the failure population of greatest interest for specification-aligned repair.

2.0.4. 2.4 POSITION RELATIVE TO PRIOR WORK

Relative to the above literature, the present work contributes (1) an explicit four-condition verification of the EvalPlus failure set as a reproducible data source for repair experiments, (2) the first three-condition McNemar design isolating specification-aligned repair from blind reprompting on EvalPlus’s own semantic failure set under its augmented test suite, and (3) pre-registration of comparative predictions before mechanism experiment execution.

3. Methodology

3.0.1. 3.1 OVERVIEW

The observation that static analysis fires on at most 32.4% of EvalPlus semantic failures motivates a repair oracle that addresses the dominant failure mode directly. The specification-aligned repair oracle provides the structured information necessary for targeted algorithmic repair through three orthogonal components: intent anchoring via the problem docstring, behavioral gap specification via a failing test input/output pair, and deviation detection via the model’s actual incorrect output. The design is referred to as the *specification triple*.

The methodology proceeds in two stages: (1) an ex-

istence verification stage that establishes the data infrastructure for reproducible repair experiments, and (2) a pre-registered mechanism comparison stage that tests whether the triple outperforms blind reprompting. This paper reports the first stage; the second stage is pre-registered and awaiting execution.

3.0.2. 3.2 THE SPECIFICATION TRIPLE (CONDITION C)

Component rationale. The three components of the triple each address a distinct informational deficit that prevents targeted algorithmic repair:

1. *Docstring re-anchoring.* Including the problem docstring’s formal intent re-anchors the model on the intended algorithm, preventing regeneration of the same incorrect solution. Haeri and Ghelichi (2026) identify specification grounding as the primary driver of a 38-percentage-point improvement in correct code generation; Dai et al. (2025) report severe performance degradation when docstrings are removed from repair feedback.
1. *Input/output counterexample.* The failing test’s input and expected output provide a concrete behavioral gap — a counterexample specifying exactly where the model’s behavior diverges from the specification. Kong et al. (2024) demonstrate that contrastive input/output pairs outperform single failure messages (143 vs. 124 bugs fixed on Defects4J).
1. *Deviation detection.* The model’s actual incorrect output enables comparison against the expected output, allowing the model to identify the specific algorithmic error. Iscan (2026) confirms that content including actual output drives repair improvement (code plus facts: +18 percentage points over bare code, $p = 0.00042$).

Prompt construction (Condition C):

[Original problem prompt]

--- Specification Context ---

DOCSTRING (formal intent): [problem docstring]

FAILING TEST:

Input: [plus_input[0] from EvalPlus API]

Expected output: [plus_output[0] from EvalPlus API]

ACTUAL MODEL OUTPUT:

[stored incorrect solution from solutions_cache.jsonl]

--- End Specification Context ---

Please repair the solution so that it passes all test cases.

3.0.3. 3.3 EXPERIMENTAL CONDITIONS

Three conditions are defined for the mechanism experiments:

Condition	Description	New API Calls Required
A — Round-0 baseline	GPT-4o-mini round-0 output from h-e1 Run 2 archive	0
B — Blind reprompting	Original prompt plus "The above solution is incorrect. Please try again."	128
C — Specification-aligned repair	Original prompt plus specification triple	128

Condition B is designed as a placebo control following Iscan (2026): it provides re-exposure to the task without any informative error context, isolating the effect of specification content from the effect of additional sampling.

3.0.4. 3.4 DATA INFRASTRUCTURE

Source. The 134-problem failure set is drawn from the h-e1 Run 2 archive (stored at `_archive/20260822T150921\_routing\_recovery/h-e1/results/`). Four conditions are verified before any mechanism API calls are made:

- **C1 — Failure ID confirmation:** 134 failure task identifiers confirmed (34 HumanEval+ and 100 MBPP+) from archived evaluation result files.
- **C2 — Stored solution coverage:** All 134 failure task identifiers have stored GPT-4o-mini incorrect solutions in `solutions\_cache.jsonl` (542 total entries; 134 unique task identifiers covered).
- **C3 — EvalPlus API accessibility:** All 134 failure task identifiers are accessible via `get\_human\_eval\_plus()` and `get\_mbpp\_plus()` API calls (EvalPlus version 0.3.1).
- **C4 — Deterministic test selection:** The `plus\_input` field is populated for tested task identifiers (sample task HumanEval/10: 780 augmented test cases), confirming that `plus\_input[0]` provides a deterministic, non-empty failing test for prompt construction.

Six tasks with empty `plus\_fail\_tests` fields in the h-e1 archive are excluded from the mechanism experiment working set: HumanEval/143, Mbbp/725, Mbbp/726, Mbbp/765, Mbbp/805, and Mbbp/809. These six records carry `plus\_status = 'fail'` but empty `plus\_fail\_tests` lists, most likely due to runner timeout or sandbox exception during

h-e1 Run 2 evaluation. The remaining 128-task working set represents 95.5% recovery of the original failure set.

Verification script. The existence verification is implemented as a Python script (`h-e1-v2/code/verify\_h\_e1\_v2.py`, approximately 80 lines) with five pytest integration tests. The script runs in approximately 2 seconds on CPU without GPU utilization, consistent with a pure data verification task.

3.0.5. 3.5 STATISTICAL DESIGN

Primary prediction (P1, hypothesis h-m1). One-tailed McNemar’s test on Condition B versus Condition C fix outcomes across 128 problems. Success criterion: $p < 0.05$ in the direction $C > B$. Yates’ continuity correction is applied if any cell count falls below 5; Fisher’s exact test is the fallback if total discordant pairs fall below 25.

Secondary prediction (P2, hypothesis h-m2). One-tailed McNemar’s test for Condition C versus Condition A; fix rate threshold of at least 15%.

Exploratory prediction (P3, hypothesis h-c1). Stratified fix rates by benchmark type (34 HumanEval+ versus 100 MBPP+) with Fisher’s exact test; directional comparison only.

Model configuration (pre-registered). GPT-4o-mini via OpenAI API, temperature = 0.2, seed = 42.

4. Experimental Setup

Three research questions structure the experiments:

RQ1: Is the 134-problem EvalPlus failure set fully recoverable from archive for reproducible repair experiments?

RQ2 (pre-registered): Does specification-aligned repair (Condition C) achieve significantly higher round-1 pass@1 than blind reprompting (Condition B)?

RQ3 (pre-registered, exploratory): Does the HumanEval+ fix rate exceed the MBPP+ fix rate under Condition C?

4.0.1. 4.1 DATASET

All 134 failures are semantic or algorithmic errors: ruff and mypy fire on at most 32.4% of HumanEval+ failures and 16.0% of MBPP+ failures, leaving 67.6% and 84.0%, respectively, with no actionable static analysis signal.

Benchmark	Failures	Proportion
HumanEval+	34	25.4%
MBPP+	100	74.6%
Total	**134**	**100%**

Working set. 128 tasks (6 excluded for empty `plus\_fail\_tests`; 95.5% recovery rate).

4.0.2. 4.2 BASELINES

Condition	Method	Rationale
A	h-e1 Run 2 results (no new API calls)	Failure population ground truth
B	Fixed reprompt: “The above solution is incorrect. Please try again.”	Placebo control — re-exposure without informative content
C	Specification triple (this work)	Tests value of structured semantic gap description

4.0.3. 4.3 EVALUATION METRICS

Primary: Round-1 pass@1 — binary pass or fail per problem; all EvalPlus augmented tests must pass, not only the prompted failing test.

Secondary: Fix rate (proportion of working-set problems fixed under Condition C; threshold 15%), token efficiency ratio (fix rate per 1000 additional input tokens relative to Condition A).

4.0.4. 4.4 IMPLEMENTATION DETAILS

- Model: GPT-4o-mini (OpenAI API), temperature = 0.2, seed = 42
- Test selection: `plus\_input[0]` (deterministic, pre-registered)
- Evaluation oracle: EvalPlus augmented test suite (all tests must pass for a pass verdict)
- EvalPlus version: 0.3.1
- Estimated cost for mechanism experiments: approximately \$0.05 USD (256 API calls total)
- Hardware: 5× NVIDIA H100 NVL (not used for existence verification; mechanism experiments require CPU and network only)

5. Results

5.0.1. 5.1 EXISTENCE VERIFICATION (RQ1)

All four conditions pass; no conditions fail.

Table 1. h-e1-v2 Existence Verification Results

Five pytest integration tests pass in 2.32 seconds. The 4/4 condition pass rate confirms that Conditions B and C prompts can be constructed for all 128 working-set tasks without new baseline API calls.

Condition	Metric	Target	Actual	Status
C1: Failure IDs	Total failures	134	134	PASS
C1: Failure IDs	HumanEval+	34	34	PASS
C1: Failure IDs	MBPP+	100	100	PASS
C2: Stored solutions	Cache coverage	134/134	134/134	PASS
C3: EvalPlus API	HumanEval+ accessible	34/34	34/34	PASS
C3: EvalPlus API	MBPP+ accessible	100/100	100/100	PASS
C4: Test selection	'plus_input' count (sample task)	> 0	780	PASS

5.0.2. 5.2 STATIC ANALYSIS ORACLE CHARACTERIZATION

Table 2. Static Analysis Fire Rates on EvalPlus Failures (h-e1 Run 2)

Benchmark	SA fire rate	Problems with no SA signal
HumanEval+ (n = 34)	32.4% (11/34)	67.6% (23/34)
MBPP+ (n = 100)	16.0% (16/100)	84.0% (84/100)

These rates quantify the scope of the problem motivating specification-aligned repair. Between two-thirds and five-sixths of EvalPlus failures receive no actionable static analysis diagnostic signal, confirming that the dominant failure mode is semantic rather than syntactic.

5.0.3. 5.3 ARCHIVE DATA INTEGRITY FINDING

Inspection of the h-e1 Run 2 archive reveals that 6 of 134 failure records carry `plus_status = 'fail'` but `plus_fail_tests = []`. The six affected task identifiers are HumanEval/143, Mbpp/725, Mbpp/726, Mbpp/765, Mbpp/805, and Mbpp/809. The most probable cause is test runner timeout or sandbox exception during h-e1 Run 2 evaluation, which stored the fail status flag but did not capture the failing test inputs. This represents an isolated data quality gap rather than a systematic failure of the archive.

The `solutions_cache.jsonl` file, by contrast, covers all 134 failure task identifiers completely (542 total entries), confirming that stored incorrect solutions are available for all tasks. Condition B and Condition C prompt construction uses `solutions_cache.jsonl` for the model’s prior incorrect output and the EvalPlus API’s `plus_input` field for the failing test input/output pair; both sources are complete for the 128-task working set.

The 6-task exclusion yields a 128-task working set representing 95.5% recovery of the original failure population. This exclusion is principled — the six tasks can-

not support Condition C prompt construction without fresh EvalPlus evaluation calls — and does not affect the statistical adequacy of the working set.

5.0.4. 5.4 PRE-REGISTERED PREDICTIONS

Table 3. Pre-Registered Prediction Status

INCONCLUSIVE indicates that predictions are registered before data collection; mechanism experiments h-m1, h-m2, and h-c1 have not been executed. The existence gate PASS (4/4 conditions) unblocks all three mechanism experiments.

5.0.5. 5.5 STATISTICAL POWER FOR PLANNED EXPERIMENTS

At $n = 128$ and an expected scenario of approximately 25% fix rate under Condition C and 10% under Condition B, approximately 24 discordant pairs are expected. One-tailed McNemar power at $\alpha = 0.05$ is approximately 80% under this scenario. If discordant pairs fall below 25, the pre-planned fallback is Fisher’s exact test on the same 2×2 table, as specified in the pre-registered design.

6. Discussion

6.0.1. 6.1 INTERPRETATION OF INFRASTRUCTURE FINDINGS

The EvalPlus failure set constitutes a reproducible foundation for repair research. The four-condition verification protocol confirms that all components required for mechanism experiment execution — failure identifiers, stored incorrect solutions, EvalPlus API access, and deterministic test selection — are available from the h-e1 Run 2 archive. Any research group with access to the archive and EvalPlus 0.3.1 can reproduce the infrastructure without additional baseline API calls.

Static analysis is the wrong oracle for EvalPlus semantic failures. The 16–32% fire rate across both

Prediction	Statement	Test	Success Criterion	Status
P1	Condition C achieves higher round-1 pass@1 than Condition B	h-m1 (McNemar, one-tailed)	$p < 0.05$	INCONCLUSIVE
P2	Condition C achieves higher pass@1 than Condition A	h-m2 (McNemar, one-tailed)	$p < 0.05$; fix rate $\geq 15\%$	INCONCLUSIVE
P3	HumanEval+ fix rate exceeds MBPP+ fix rate under Condition C	h-c1 (Fisher's exact, directional)	Directional	INCONCLUSIVE

benchmark subsets means that the majority of failures are opaque to the standard tool-assisted repair pipeline. This falsification of the static analysis oracle, established empirically in h-e1 Run 2, is the primary motivation for specification-aligned repair as a replacement oracle. Specification-aligned repair addresses the failure mode directly: it provides the intent, behavioral gap, and deviation signal that static analysis cannot supply.

The h-e1 to h-e1-v2 trajectory illustrates appropriate verification target selection. The initial h-e1 verification script tested `plus\_fail\_tests` completeness and failed because 6/134 records had empty fields (gate: FAIL, 5/6 checks passing). The h-e1-v2 redesign correctly identified that `solutions\_cache.jsonl` — not `plus\_fail\_tests` — is the relevant data source for prompt construction in Conditions B and C, because the model's incorrect output is stored in the solutions cache and the failing test input/output pair is obtained via the EvalPlus API at experiment time. This redesign represents appropriate hypothesis refinement in response to a verification failure, not abandonment of the core research direction.

6.0.2. 6.2 LIMITATIONS

Mechanism experiments not executed. Predictions P1, P2, and P3 are pre-registered but unmeasured. All comparative claims — whether specification-aligned repair outperforms blind reprompting — are INCONCLUSIVE. This paper establishes infrastructure and registers the design; the mechanism experiments are the primary next step.

Six-task exclusion ($n = 128$, not $n = 134$). The principled exclusion of 4.5% of the failure set for missing archive data reduces the working set modestly. Statistical power analysis indicates that $n = 128$ is adequate for the expected effect size range (15–40% fix rate under Condition C). The exclusion is documented and the specific task identifiers are reported.

Single model and temperature. All experiments use GPT-4o-mini at temperature = 0.2, seed = 42. Generalization to other models, model scales, temperatures, and sampling configurations is not addressed and is left for future work. The assumption that GPT-4o-mini can leverage structured specification context to improve repair performance over blind reprompting

remains unverified pending mechanism experiment execution.

Component ablation deferred. The specification triple is evaluated as an integrated oracle. Individual component attribution — whether docstring re-anchoring, input/output counterexample, or actual output deviation detection is the primary driver of any repair improvement — requires a full 7-condition ablation. This ablation is deferred to future work; the present design tests only the integrated triple against a blind reprompting placebo.

Single-round repair. Only round-1 repair is evaluated under conditions B and C. Multi-round repair with updated specification context per round is a tractable extension deferred to future work.

6.0.3. 6.3 IMPLICATIONS FOR REPRODUCIBILITY METHODOLOGY

The four-condition existence verification protocol developed for this work — checking failure identifier availability, stored solution coverage, API accessibility, and deterministic test selection — is a reusable template applicable to any repair experiment built on an archived LLM evaluation failure set. Pre-registration of quantitative predictions before mechanism data collection prevents hypothesis drift and guards against post-hoc analysis bias. Together, the verified infrastructure and the pre-registered design constitute a methodological contribution to reproducibility in LLM code repair evaluation that is independent of the mechanism experiment outcomes.

7. Conclusion

EvalPlus semantic failures — the code generation errors that survive round-0 evaluation by GPT-4o-mini and evade static analysis detection — represent the hardest and most consequential target for LLM repair research. Static analysis fires on at most 32.4% of these failures; the remaining 67–84% receive no actionable diagnostic signal from the standard tool-assisted pipeline. Blind reprompting provides no additional information to the model, offering limited prospects for improvement on a failure population characterized by algorithmic rather than syntactic errors.

This paper proposes specification-aligned repair — a structured oracle providing the model's docstring-

based formal intent, a failing test input/output counterexample, and the model’s actual incorrect output — as a targeted alternative. The data infrastructure for evaluating this approach has been verified: 134 GPT-4o-mini EvalPlus failure identifiers are confirmed, 134 stored incorrect solutions are present in the archive, the EvalPlus API is accessible for all 134 task identifiers, and deterministic test selection via `plus\_input[0]` is confirmed. Six tasks with empty archived failing-test records define a 128-task working set. All four existence conditions pass; 5/5 pytest tests pass in 2.32 seconds.

Three predictions are pre-registered before mechanism data collection: that specification-aligned repair outperforms blind reprompting (P1, McNemar one-tailed $p < 0.05$), that it achieves a fix rate of at least 15% above the round-0 baseline (P2), and that HumanEval+ fix rate exceeds MBPP+ fix rate under specification-aligned repair (P3, exploratory). Mechanism experiments h-m1, h-m2, and h-c1 are unblocked by the existence gate PASS and are the designated next step.

The infrastructure and pre-registration reported here are independently valuable: they ensure that when the mechanism experiments execute, the data provenance is verified, the comparison is fair, and the predictions were committed before the results were known.

7.0.1. FUTURE WORK

Component ablation. Removing one element of the specification triple at a time — docstring only, input/output pair only, actual output only — will determine the primary driver of any repair improvement and is the most important follow-on experiment.

Stronger model evaluation. If P1 indicates $C \approx B$ for GPT-4o-mini, testing with GPT-4o or Claude 3.5 Sonnet is the appropriate next step before concluding that the approach fails generally.

Scope extensions. Recovering the 6 excluded tasks via fresh EvalPlus evaluation calls (estimated at approximately one hour of engineering effort), multi-round repair with per-round context updates, and evaluation on LiveCodeBench for training contamination analysis are immediately tractable extensions.

References

Akli, A., Akli, M., Richter, C., Papadakis, M., and Traon, Y. L. From failing to passing: Evolving natural language prompt optimization rules for LLM code generation. *arXiv preprint arXiv:2607.05121*,

2026. doi: 10.48550/arXiv.2607.05121.

Arimbur, J. J. How many tries does it take? iterative self-repair in LLM code generation across model scales and benchmarks. *arXiv preprint arXiv:2604.10508*, 2026. doi: 10.48550/arXiv.2604.10508.

Clarke, E., Grumberg, O., Jha, S., Lu, Y., and Veith, H. Counterexample-guided abstraction refinement for symbolic model checking. In *Journal of the ACM*, 2003.

Dai, D., Liu, M., Li, A., Cao, J., Wang, Y., Wang, C., Peng, X., and Zheng, Z. FeedbackEval: A benchmark for evaluating large language models in feedback-driven code repair tasks. *arXiv preprint arXiv:2504.06939*, 2025. doi: 10.48550/arXiv.2504.06939.

Haeri, A. and Ghelichi, M. Specification grounding drives test effectiveness for LLM code. *arXiv preprint arXiv:2607.06636*, 2026.

Iscan, M. Falsification, not exposure: An internally preregistered placebo-controlled decomposition of self-repair feedback in frozen small code models. *arXiv preprint arXiv:2606.31511*, 2026. doi: 10.48550/arXiv.2606.31511.

Kong, J., Xie, X., Cheng, M., Liu, S., Du, X., and Guo, Q. ContrastRepair: Enhancing conversation-based automated program repair via contrastive test case pairs. *ACM Transactions on Software Engineering and Methodology*, 34:1–31, 2024. doi: 10.1145/3719345.

Kulsum, U., Zhu, H., Xu, B., and d’Amorim, M. A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. In *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*, 2024. doi: 10.1145/3664646.3664770.

Liu, J., Xia, C., Wang, Y., and Zhang, L. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.52202/075280-0943.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Welleck, S., Majumder, B. P., Gupta, S., Yazdanbakhsh, A., and Clark, P. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.48550/arXiv.2303.17651.

Wang, K., Mao, B., Jia, S., Ding, Y., Han, D., Ma,
T., and Cao, B. SGCR: A specification-grounded
framework for trustworthy LLM code review. In *Pro-
ceedings of the 40th IEEE/ACM International Con-
ference on Automated Software Engineering (ASE)*,
pp. 3778–3783, 2025. doi: 10.1109/ASE63991.2025.
00315.